TASK-10: Build a Student CRUD application using Spring Boot.

AIM: Build a Spring Boot application to perform CRUD (Create, Read, Update, Delete) operations on a Student1 table using JPA annotations**.**

ALGORITHM:

1. Start the Spring Boot application
2. Connect to MySQL database
3. Receive HTTP request (POST/GET/PUT/DELETE)
4. Controller handles the request
5. Service processes business logic
6. Repository interacts with database using JPA
7. Perform Create / Read / Update / Delete operation
8. Return response to client
9. Stop

OUTPUT1: performing created operation

http://localhost:8080/students

Save | Share

GET http://localhost:8080/students Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Keeru",
3   "email": "keeru@gmail.com"
4 }
```

Body 200 OK 410 ms 268 B

{} JSON Preview Visualize

```
1  [
2    {
3      "email": "sthtavarthi@gmail.com",
4      "id": 1,
5      "name": "keerthana"
6    },
7    {
8      "email": "keeru@gmail.com",
9      "id": 2,
10     "name": "Keeru"
11   }
12 ]
```

OUTPUT2: performing read operation

http://localhost:8080/students/1

Save | Share

PUT http://localhost:8080/students/1 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "keerthana",
3   "email": "thtavarthi@gmail.com"
4 }
```

Body 200 OK 772 ms 214 B

{} JSON Preview Visualize

```
1 {
2   "email": "thtavarthi@gmail.com",
3   "id": 1,
4   "name": "keerthana"
5 }
```

OUTPUT3: performing update operation

http://localhost:8080/students

Save | Share

GET http://localhost:8080/students Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Keeru",
3   "email": "keeru@gmail.com"
4 }
```

Body 200 OK 410 ms 268 B

{} JSON Preview Visualize

```
1  [
2      {
3          "email": "sthtavarthi@gmail.com",
4          "id": 1,
5          "name": "keerthana"
6      },
7      {
8          "email": "keeru@gmail.com",
9          "id": 2,
10         "name": "Keeru"
11     }
12 ]
```

OUTPUT4: performing delete operation

http://localhost:8080/students/1

Save | Share

DELETE http://localhost:8080/students/1 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "keerthana",
3   "email": "thtavarthi@gmail.com"
4 }
```

Body 200 OK 332 ms 184 B

Raw Preview Visualize

```
1 Deleted successfully
```

Result:

Student data is successfully stored, retrieved, updated, and deleted using REST APIs. The application runs correctly with Spring Boot and JPA connected to MySQL